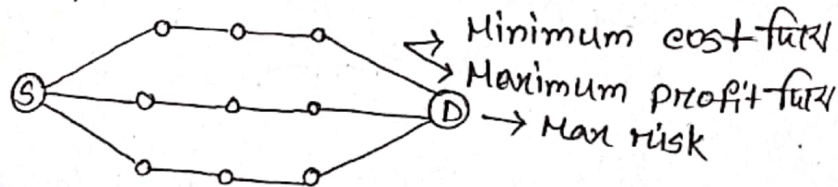


(segment-4)

Greedy Algorithm:

It is an algorithm that finds a solution to problem in the shortest time possible.

- ⇒ It involves making the locally optimal choice at each stage with the hope of finding the global optimum.
- ⇒ It is easy to implement and understand
- ⇒ Feasible solution
- ⇒ Optimal solution (Min cost, Max profit, Min Risk)
- ⇒ Selection criteria
- ⇒ For optimization problem it can be used (not always)
- ⇒ Top-down approach



Greedy Strategy:

- ⇒ Start with a solution to a small subproblem
- ⇒ Build up to a solution to the whole problem
- ⇒ Make choices that look good in the short term.

Advantage: ⇒ Greedy algorithm work fast when they work.

- ⇒ Simple algorithm
- ⇒ Easy to implement

Disadvantage: $\Rightarrow$ Greedy algorithms don't always work
(short term solutions can be disastrous
in the long term).

$\Rightarrow$ Hard to prove correct.

Application: ① knapsack problem

② Minimum spanning tree

③ Activity selection problem

④ Huffman coding

⑤ Dijkstra's algorithm

Optimal solutions: 1. change making for 'normal' coin
denominations.

2. minimum spanning tree (MST)

3. Single source shortest paths

4. Simple scheduling problems

5. Huffman codes

Approximations: 1. Travelling salesman problem (TSP)

2. knapsack problem

3. Other combinatorial optimization problems

Greedy characteristics:

1. Local optimality
2. Not back tracking
3. Not always the best

Greedy Algorithm Vs Dynamic programming

Greedy Algorithm	Dynamic Programming
1. we make our decisions based on the best current situation.	1. we select individually in every step.
2. There is no assurance of obtaining the optimal solution.	2. we can get the assurance of an optimal solution.
3. Follows top down approach.	3. Follows bottom up and top down approach.
4. More efficient	4. Less efficient
5. Faster, simpler	5. Slower, More complex

Activity Selection Problem:

Select maximum number of activities to solve by a single person.

- ① Sorting the finishing time.
- ② Find compatible activity and add to solution.

Input: Set S of n activities, $a_1, a_2, \dots, a_n$

s_i = start time of activity i

f_i = finish time of activity i .

Output: Subset A of maximum number of compatible activities.

Two activities are compatible, if their intervals don't overlap.

Example:

	a_1	a_2	a_3	a_4	a_5	a_6	a_7
s_i	2	1	4	1	9	8	5
f_i	5	8	7	3	11	10	9

- ① Sorting the finishing time
- ② Find compatible activity and add to solution

Soln:

Sorting the finishing time

	a_4	a_1	a_3	a_2	a_7	a_6	a_5
s_i	1	2	4	1	5	8	9
f_i	3	5	7	8	9	10	11

$$A = \{a_4, a_3, a_6\}$$

Optimal substructure:

□ S_{ij} denote the set of activities between a_i and a_j and compatible with a_i and a_j .

→ Activities start after a_i finishes and finish before a_j starts.

$$S_{ij} = \{a_k \in S : f_i \leq s_k < f_k \leq s_j\}$$

□ we define a_0 and a_{n+1} such that $f_0 = 0$ and $s_{n+1} = \infty$.

→ $S = S_{0,n+1}$ for $0 \leq i, j \leq n+1$.

→ Assume that activities are sorted in increasing order of finish time.

$$f_0 \leq f_1 \leq f_2 \leq \dots \leq f_n \leq f_{n+1}$$

□ A_{ij} denote an optimal solution to S_{ij} for $i \leq j$

→ $S_{ij} = \emptyset$ if $i > j$

→ if A_{ij} includes a_k , $A_{ij} = A_{ik} \cup \{a_k\} \cup A_{kj}$

Recursive solution:

→ $c[i, j]$: The number of activities in A_{ij} .

$$c[i, j] = \begin{cases} 0 & \text{if } S_{ij} = \emptyset \\ \max_{\substack{i < k < j \\ a_k \in S_{ij}}} \{c[i, k] + c[k, j] + 1\} & \text{if } S_{ij} \neq \emptyset \end{cases}$$

Pseudocode:

greedy-activity (s, f)

{

$n \leftarrow \text{length}[s]$

$A \leftarrow \{a_1\}$

$j \leftarrow \{1\}$

for $i \leftarrow 2$ to n

do if $s_i \geq f_j$

then $A \leftarrow A \cup \{i\}$

$j \leftarrow i$

return A ;

}

Time complexity: $O(n \log n) + O(n)$

$\rightarrow O(n \log n)$

knapsack problem: (greedy method)

Example-1

Objects: 1 2 3 4 5 6 7

$n = 7$

Profit : 12 5 16 7 9 11 6

$m = 15$

Weight : 3 1 4 2 9 4 3



$\frac{P}{W}$:

4	5	4	3.5	1	2.75	2
---	---	---	-----	---	------	---



Objects	profits	weight	Remaining weight
2	5	1	$15 - 1 = 14$
1	12	3	$14 - 3 = 11$
3	16	4	$11 - 4 = 7$
4	7	2	$7 - 2 = 5$
6	11	4	$5 - 4 = 1$
7	$6 \times \frac{1}{3} = 2$	1	$1 - 1 = 0$
53		15	

Max profit

Min profit

- We are given n objects and a knapsack. Object i has a weight w_i and the knapsack has a capacity M .
 If a fraction x_i , $0 \leq x_i \leq 1$ of object i is placed into the knapsack then a profit of $P_i x_i$ is earned.
 The objective is to obtain a filling of the knapsack that maximizes total weight of all chosen objects to be at most M .

Example:

Object	1	2	3
Profit	25	24	15
weight	18	15	10

19	19
9	9

Capacity $(M) = 20$

Soln: P/w for objects, $Ob_1 = \frac{25}{18} = 1.3$
 $Ob_2 = \frac{24}{15} = 1.6$
 $Ob_3 = \frac{15}{10} = 1.5$

[এখানে $\frac{P}{w}$ এর ব্যৱহাৰ হয়, যাৰ P/w বান্ধি এখানে অৰ্থে Count কৰা হয়, আৰু decreasing way ত গিৱে M ফিলু কৰা হয়।]

$\therefore 24 + \frac{5}{10} \times 15 = 31.5$

[Capacity count কৰাৰে weight but result show হয় Profit.]

Pseudo code:

for $i = 1$ to n
 calculate P/W] $\rightarrow O(n)$

Sort objects in decreasing
 order of P/W ratio] $\rightarrow O(n \log n)$

for $i = 1$ to n
 if $(M > 0 \text{ and } w_i \leq M)$] $\rightarrow O(n)$
 $M = M - w_i$
 $P = P + p_i$
 if $(M > 0)$
 $P = P + p_i \times (\frac{M}{w_i})$

Weight	10	20	30	40
Profit	60	100	120	150

$$O(n) + O(n \log n) + O(n) = O(n \log n)$$

Knapsack 0/1 can not be solved by greedy strategy.

30
20

\$120

\$100

20
10

\$100

\$60

30
10

\$120

\$60

Total

= \$220

= \$160

= \$180

The value per pound of item 1 is 6 dollars per pound, which is greater than the value per pound of either item 2 (5 dollars per pound) or item 3 (4 dollars per pound). Therefore, the greedy strategy

would take item 1 first. As can be seen from the case analysis in above figure, the optimal solution takes items 2 and 3, leaving 1 behind. The two possible solutions that involve item 1 are both suboptimal.

0/1 knapsack problem

Question: why greedy algorithm fail here? Give an example.

* [0/1 knapsack (greedy algorithm fail ex)]

As an example, we have 12kg sack and three objects: Mobile, tab, laptop according to greedy method.

	01	02	03
P	20	25	60
w	2	4	8
$\frac{P}{w}$	10	6.2	7.5

obj 3	w = 8
obj 1	w = 2
	12kg
	10kg

In sack we can put object 1 ($\frac{P}{w} - \max$), 2nd we can put obj 3 ($7.5 \approx 8$), ($8+2=10$) = left weight is 2. If we put obj 2 and obj 3 ($8+4=12$) which can easily fill the sack that's why greedy method will fail here.

According to Dynamic programming,

2^n , n = object number

$$2^3 = 8$$

obj 1	obj 2	obj 3
0	0	0 X
0	0	1 $\rightarrow$ Profit = 60
0	1	0 $\rightarrow$ Profit = 25
0	1	1 $\rightarrow$ Profit = $(60+25) = 85$
1	0	0 $\rightarrow$ Profit = 20
1	0	1 $\rightarrow$ Profit = $80 = (20+60)$
1	1	0 $\rightarrow$ Profit = $45 = (20+25)$
1	1	1 X

Here,

000 and 111 is not possible.

Total profit is high which is 85 if we fill up the sack with obj 2 and obj 3.

00	01	10	11
			1
	1	2	10
0.5	0.5	01	9

Huffman Coding:

- ⇒ For compressing data (sequence of characters)
- ⇒ widely used
- ⇒ Very efficient (saving 20-90%)
- ⇒ use a table to keep frequencies of occurrence of characters.
- ⇒ Output binary string.

There are two method. 1. Fixed length 2. Huffman coding / Variable length code

Example:

	A	B	C	D	E	F
Frequency	45	13	12	16	9	6

(Frequency table ની માર્ગલ count રજા વિશે શરૂ)

Soln: (Method -1)

Fixed length = 2^n , n = bit number

$2^1 = 2$; it represent 2 letter

$2^2 = 4$; it represent 4 letter

$2^3 = 8$; it represent 8 letter

So, 6 (A, B, C, D, E, F) letter length can be represented by 3 bit

$$\begin{aligned}\text{Total bit} &= (45 + 13 + 12 + 16 + 9 + 6) \times 3 \\ &= 100 \times 3 \\ &= 300\end{aligned}$$

	A	B	C	D	E	F
Frequency	45	13	12	16	9	6
Fixed length	000	001	010	011	100	101

Fixed length Table

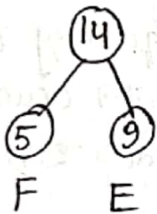
(Method - 2)

A: 45 B: 13 C: 12 D: 16 E: 9 F: 6

Sorting,

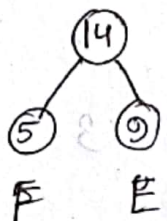
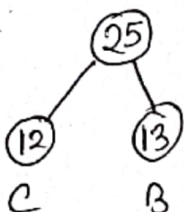
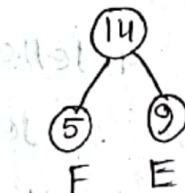
F: 6 E: 9 C: 12 B: 13 D: 16 A: 45

Now,

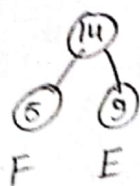


Again,

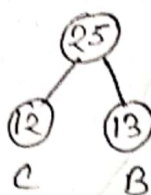
C: 12 B: 13 D: 16 A: 45



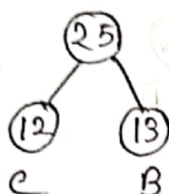
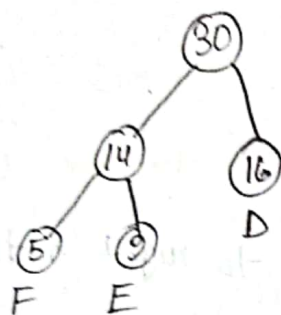
Again sorting,



D: 16

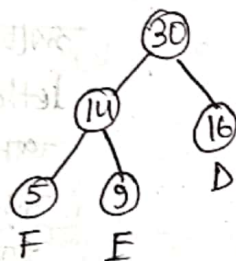
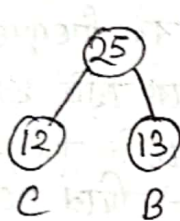


A: 45

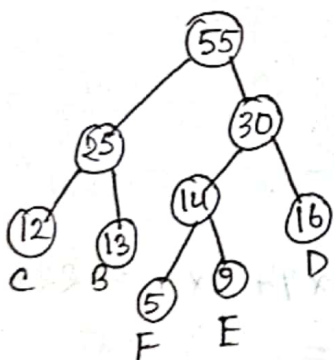


A: 45

Then,



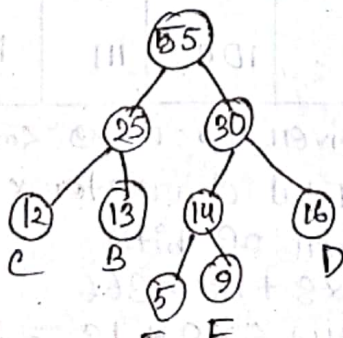
A: 45

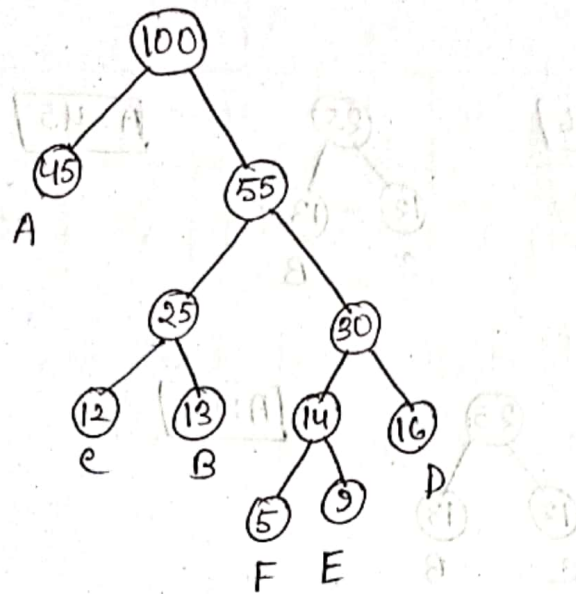


A: 45

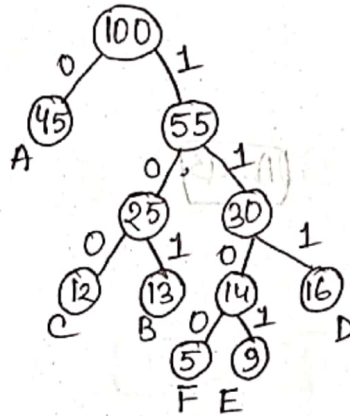
Again sorting,

A: 45





Now, assigning 0 to every left node and 1 to right node



Now, A=0

B=101

C=100

D=111

E=1101

F=1100

স্বাক্ষর বর্গ frequency letter কে ছোট বাঁ দিক আর ছোট frequency কে বড় বাঁ দিক দিয়ে represent করতে হয়,

$$\text{Total bit} = 45 \times 1 + 13 \times 3 + 12 \times 3 + 16 \times 3 + 9 \times 4 + 5 \times 4 = 224 \text{ bit}$$

	A	B	C	D	E	F
Frequency	45	13	12	16	9	5
Huffman code	0	101	100	111	1101	1100

For Actual bit (Question 9 receiver কে দিতে বললে)

For receiver = Total size + Total character x Actual number of bits +

Total number of bits

$$\text{Fixed length} = 300 + 6 \times 8 + 18 = 366$$

$$\text{Huffman codes} = 224 + 6 \times 8 + 18 = 290$$

TSP (Travelling sales person problem)

The travelling salesman problem consists of a salesman and a set of cities. The salesman has to visit each one of the cities starting from a certain one and returning to the same city.

⇒ Complete Graph: Vertices joined by a single edge.

⇒ Weighted Graph: edges carry a value.

Example:
$$g(i, s) = \min_{j \in s} \{e_{ij} + g(j, s - \{i\})\}$$

Here, $g(i, s)$ is the shortest path starting from i and going through all the vertex in s and terminate at i .

i = starting index

j = i থেকে যে নোডে যাওয়া যায় সে নোড।

s = Sum of total node.

For understand

Rules For Tree:

- ① Tree আঁকলে ক্রমবিন্যাস বা আঁকলে ক্রমবিন্যাস আছে,
- ② Tree আঁকলে নিয়ম: • যে node থেকে শুরু হবে তা node ১ হিসেবে ধরা হবে,
• ১ নং node লিখলে তা আর Repeat করতে হবে না last
• Stop স্বীকৃতি, মেনে:

২ → ৫

Possible node, ২ → ৩ → ৪ → ১ → ৫ (✓)

২ → ৩ → ২ → ৪ → ১ (X)

৩ → ৩ → ৪ → ১ → ২ (✓)

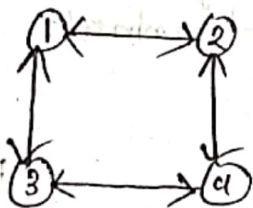
• Adjacent value নিম্নতম হতে tree to, last

stop ও আজ min value নিত হতে হবে
দরবর্তী মান রাখা করাও হয়।

• Tree এর value assign bottom up approach/

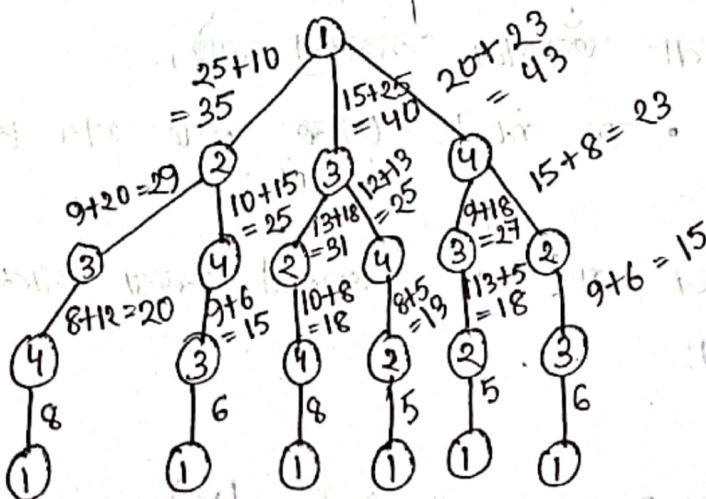
Top down approach follow করে,

Example:



1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

Soln:



Best possible way - 1 → 2 → 4 → 3 → 1.

$$\therefore g(i, s) = \min_{j \in S} \{ c_{ij} + g(j, s - \{j\}) \}$$

$$g(2, \emptyset) = 5$$

$$g(3, \emptyset) = 6$$

$$g(4, \emptyset) = 8$$

$$g(2, \{3\}) = c_{23} + g(3, \emptyset) = 9 + 6 = 15$$

$$g(2, \{4\}) = c_{24} + g(4, \emptyset) = 10 + 8 = 18$$

$$g(3, \{2\}) = c_{32} + g(2, \emptyset) = 13 + 5 = 18$$

$$g(3, \{4\}) = c_{34} + g(4, \emptyset) = 12 + 8 = 20$$

$$g(4, \{2\}) = c_{42} + g(2, \emptyset) = 8 + 5 = 13$$

$$g(4, \{3\}) = c_{43} + g(3, \emptyset) = 9 + 6 = 15$$

$$g(2, \{3, 4\}) = \begin{cases} c_{23} + g(3, \{4\}) = 9 + 20 = 29 \\ c_{24} + g(4, \{3\}) = 10 + 15 = 25 \end{cases}$$

$$g(3, \{2, 4\}) = \begin{cases} c_{32} + g(2, \{4\}) = 13 + 18 = 31 \\ c_{34} + g(4, \{2\}) = 12 + 13 = 25 \end{cases}$$

$$g(4, \{3, 2\}) = \begin{cases} c_{43} + g(3, \{2\}) = 9 + 18 = 27 \\ c_{42} + g(2, \{3\}) = 8 + 15 = 23 \end{cases}$$

$$g(1, \{2, 3, 4\}) = \begin{cases} c_{12} + g(2, \{3, 4\}) = 10 + 25 = 35 \\ c_{13} + g(3, \{2, 4\}) = 15 + 25 = 40 \\ c_{14} + g(4, \{2, 3\}) = 20 + 23 = 43 \end{cases}$$

So, the ^{best} path is $g(1, \{2, 3, 4\})$.

(segment -5)

Graph

A graph is a collection of nodes, called vertices and a collection of line segments, called lines, edges or arcs, connecting pairs of vertices.

Graph consists of 2 sets:

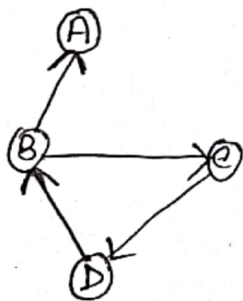
- A set of lines
- A set of vertices

Two types of graph: They are

- ① Directed Graph
- ② Undirected Graph

Directed Graph: A directed graph or digraph is a graph in which each line has a direction associated with it.

In a directed graph, the flow along an edge between 2 vertices can only follow the indicated direction.

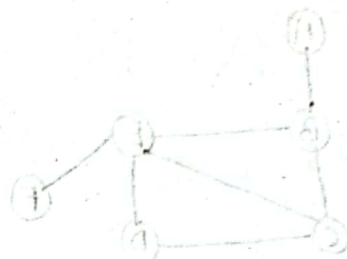
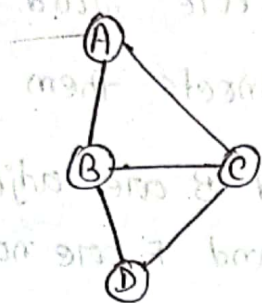


- ⊗ Simple Graph with each edge directed
- ⊗ Loop are allowed in a directed Graph.

Undirected Graph: An undirected graph is a graph in which there is no direction associated with the lines.

In an undirected graph, the flow between two

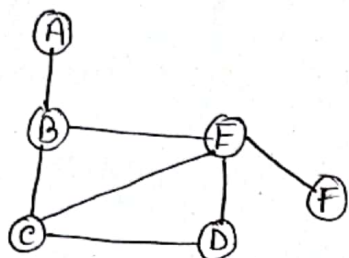
Vertices can go in either direction.



Difference between Graph and Tree :

Graph	Tree
① Graph is a non-linear data structure that can have more than one path between vertices.	① Tree is also a non-linear data structure but it has only one path between two vertices.
② Graphs can have loops	② Loops are not allowed in a tree structure.
③ Graphs can have cycle	③ cycle is not allowed
④ Graphs do not have a root node	④ Trees have exactly one root node.
⑤ Graphs have two traversal techniques. BFS and DFS.	⑤ Trees have three traversal techniques. Pre-order, in-order and post order.
⑥ Graphs follow the network model	⑥ Trees follow the hierarchical model.
⑦ Graphs are more complex	⑦ Trees are less complex.

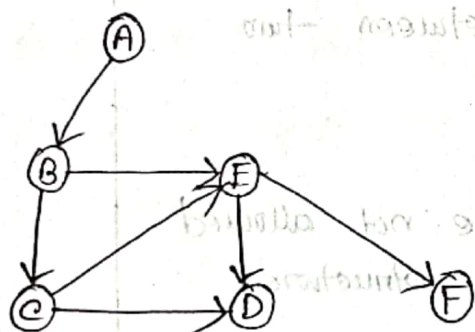
Adjacent: 2 vertices in a graph are said to be adjacent if an edge directly connects them



Here, A and B are adjacent
D and F are not adjacent

Path: A path is a sequence of vertices in which each vertex is adjacent to the next one

⇒ Vertex can repeat, edges cannot repeat



Here, A, B, C, E is a path

A, B, E, F is a path

A, B, E, C is not a path

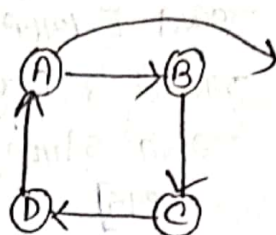
Bkz, E is adjacent to C but C is not adjacent to E (note direction)

⇒ Both directed and undirected graphs have paths

⇒ In an undirected graph, you may travel in either direction.

In a directed graph you may only travel along the given direction of the arcs.

Cycle: A cycle is a path that starts and ends with the same vertex. A single vertex is not a cycle.



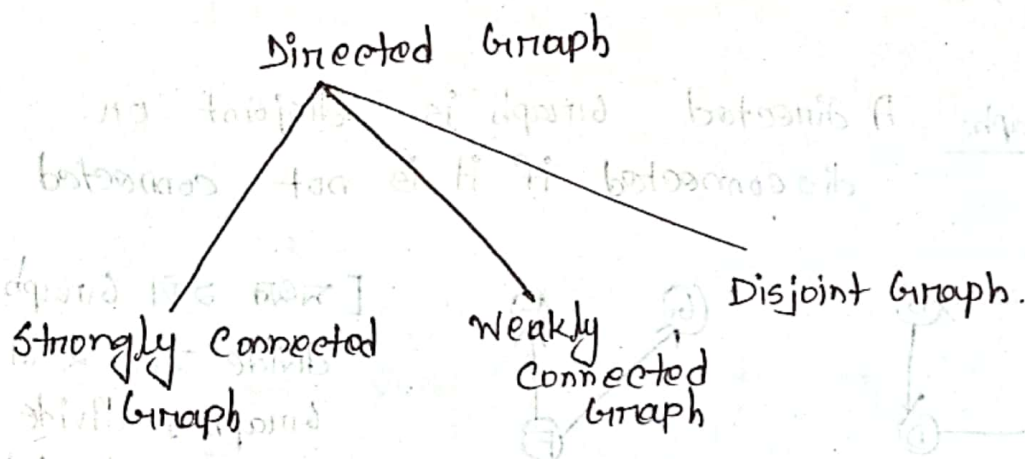
Start and end vertex is A which is ~~cycle~~ same. That's why this is cycle.

Self edge/loop: It is an arc that begins and ends at the same vertex.

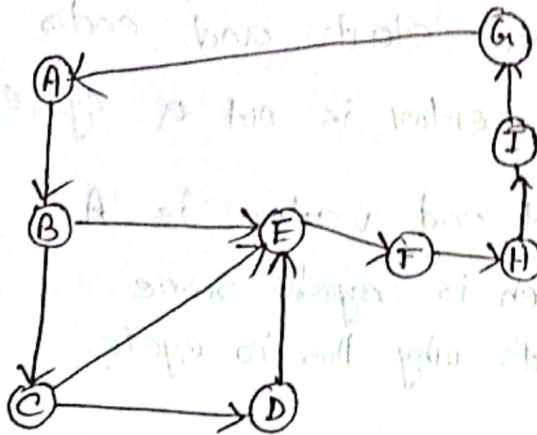


Connected Graph: Two vertices are said to be connected if there is a path between them.

A Graph is said to be connected if suppressing direction, there is a path from each vertex to every other vertex.



Strongly Connected Graph: A directed graph is strongly connected if there is a path from each vertex to every other vertex.

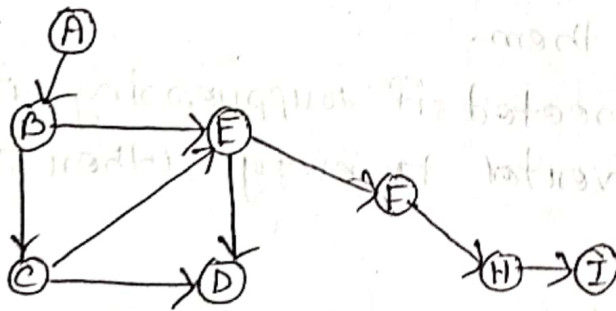


[A থেকে E তে
সমস্যা may থাকবে
-তখন E থেকে A তে
সমস্যা ৬ may থাকবে
-তখন Strongly Connected
হবে]

$A \rightarrow E (\checkmark)$

$E \rightarrow A (\checkmark)$

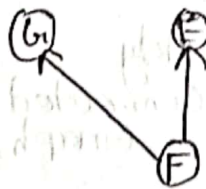
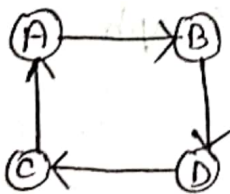
Weakly Connected Graph: A directed graph is weakly connected if at least 2 vertices are not connected.



$A \rightarrow D (\checkmark)$

$D \rightarrow A (X)$

Disjoint Graph: A directed graph is disjoint or disconnected if it is not connected



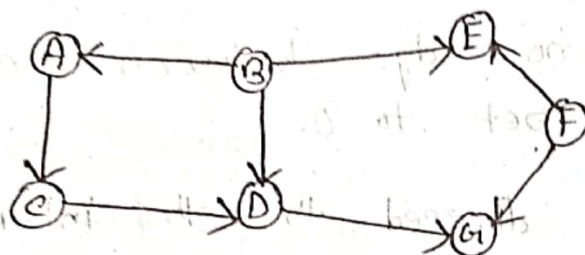
[যেহেতু ১টা Graph
divide হয় ২টা
Graph এ divide
হয় তাহলে disjoint
Graph বলে]

Degree: The degree of a vertex is the number of edges incident to it.

Outdegree: The outdegree of a vertex in a digraph is the number of arcs leaving the vertex.

Indegree: The indegree of a vertex in a digraph is the number of arcs entering the vertex.

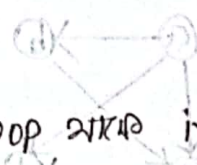
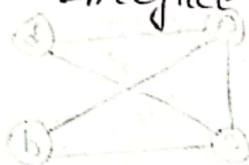
Example:



The degree of vertex B is 3

Outdegree of vertex B is 3

Indegree of vertex B is 0



[If self loop exists in and out 2 to count 2]

* If $(u, v) \in E$ then vertex v is adjacent to vertex u

* Adjacency relationship is:

→ Symmetric if G is undirected

→ Not necessarily so if G is directed

* If G is connected:

→ there is a path between every pair of vertices.

→ $|E| \geq |V| - 1$. * $|E| = |V| - 1$, then G is a tree.

1	2	3	4	5
0	1	0	0	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	1

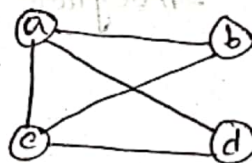
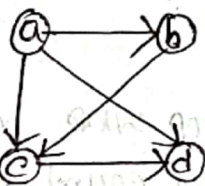
Adjacency Matrix: The adjacency matrix uses a vector (1-dimensional array) to store vertices and a matrix (2 dimensional array) to store the edges.

⇒ If 2 vertices are adjacent that is if there is an edge between them, the matrix intersect has a value of 1.

⇒ If there is no edge between them the intersect is set to 0.

⇒ If the graph is directed then the intersection chosen in the adjacency matrix indicates the direction.

For Directed Graph For undirected Graph



	a	b	c	d
a	0	1	1	1
b	0	0	1	0
c	0	0	0	1
d	0	0	0	0

A
B
C
D

Vertex
vector

	a	b	c	d
a	0	1	1	1
b	1	0	1	0
c	1	1	0	1
d	1	0	1	0

A
B
C
D

Vertex
vector

Complexity for directed and undirected - $O(V^2)$

Space: $O(V^2)$

— Not memory efficient for large graphs

Time: To list all vertices adjacent to v : $O(V)$

Time: to determine if $(u, v) \in E$: $O(1)$

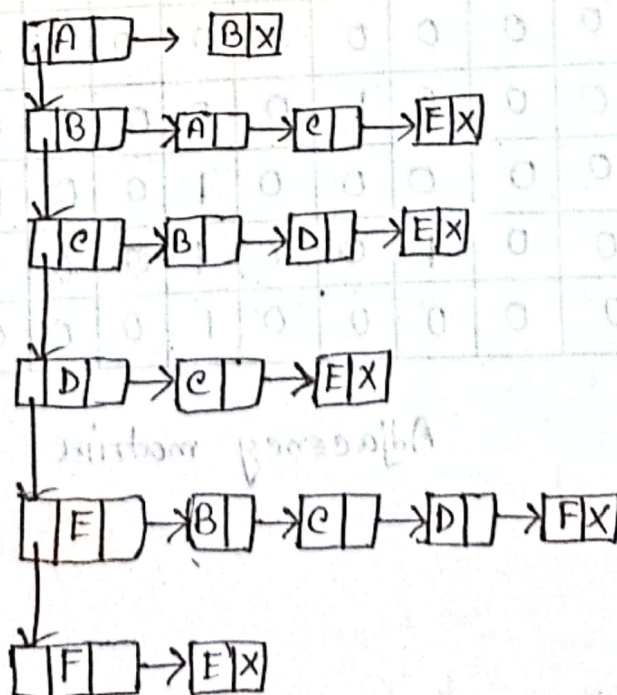
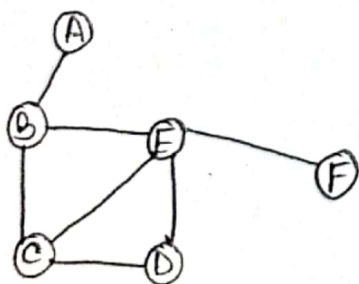
Adjacency List: Each node (vertex) has a list of which nodes (vertices) it is adjacent.

If the graph structure is unknown ahead of time, we can use linked lists to implement the vertex vector and adjacency matrix.

They are referred to as a vertex list and an adjacency list.

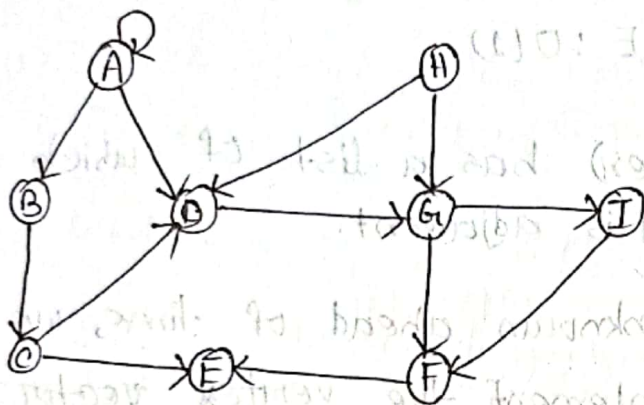
The adjacency list is a linked list containing adjacent nodes.

Example:



Vertex list Adjacency list

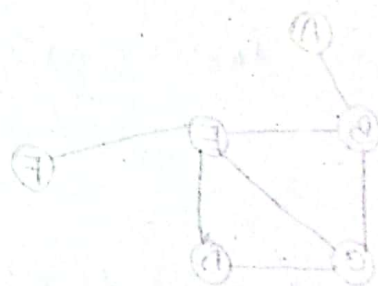
Example: Show the adjacency list and adjacency matrix representation for the graph is given below.



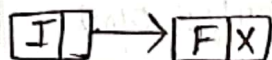
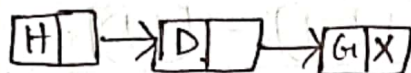
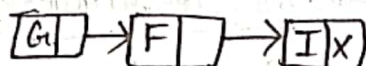
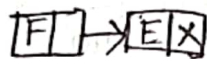
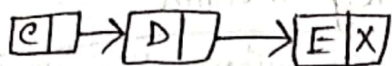
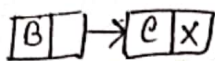
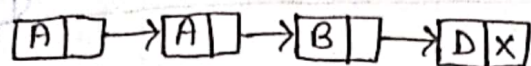
Soln:

	A	B	C	D	E	F	G	H	I
A	1	1	0	1	0	0	0	0	0
B	0	0	1	0	0	0	0	0	0
C	0	0	0	1	1	0	0	0	0
D	0	0	0	0	0	0	1	0	0
E	0	0	0	0	0	0	0	0	0
F	0	0	0	1	0	0	0	0	0
G	0	0	0	0	0	1	0	0	1
H	0	0	1	0	0	1	0	0	0
I	0	0	0	0	0	1	0	0	0

Adjacency matrix



Adjacency list:



□ For directed graph, complexity: $O(V+E)$

⊗ Sum of lengths of all adj. lists is

$$\sum_{v \in V} \text{out degree}(v) = |E|$$

↪ No. of edges leaving v

For undirected graph, complexity: $O(V+E)$

⊗ Sum of length of all adj list is

$$\sum_{v \in V} \text{degree}(v) = 2|E|$$

↪ No. of edges incident on v . Edge (u,v) is incident on vertices u and v

Difference between Adjacency List and Adjacency Matrix:

Matrix:

Adjacency List	Adjacency matrix
① It ^{only} stores information about which edges do exist in the graph	① It stores information about each possible edges (whether it exists or not)
② Good for dense graph $ E \sim O(V)$	② Good for sparse graph $ E \sim O(V ^2)$
③ Vertices adjacent to another vertex can be found quickly	③ Connectivity between two vertices can be tested quickly
④ Memory requirements: $O(V + E) = O(V)$	④ Memory requirements: $O(V + E) = O(V ^2)$
⑤ Space complexity: $O(n+m)$	⑤ Space complexity: $O(n^2)$
⑥ Fast to iterate over all edges	⑥ Slow to iterate over all edges
⑦ Hard to implement	⑦ Easy to implement

Pros: space-efficient, when graph is sparse
can be modified to support many graph variants

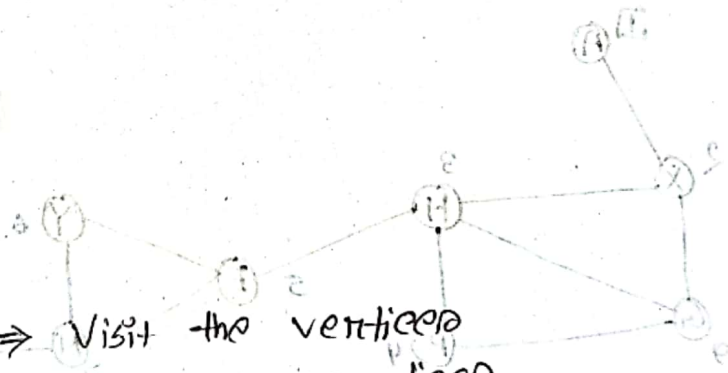
Cons: Determining if an edge $(u, v) \in G$ is not efficient.
 → Have to search in u 's adjacency list. $O(\text{degree}(u))$ time.
 → $O(V)$ in the worst case

Difference between sparse and dense Graph:

sparse Graph	Dense Graph
① A sparse Graph is a Graph in which there is a small number of edges.	① A dense graph is a graph in which there is a large number of edges.
② Few edges between nodes	② Many edges between nodes
③ Number of edges is close to n	③ Number of edges is close to n^2

Traverse Graph:

- ⇒ Visit the vertices
- ⇒ Explore the vertices



There are 2 standard graphs traversals.

① Depth First traversal (DFS)

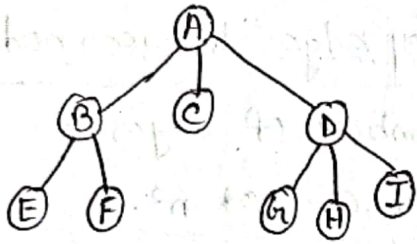
② Breadth First traversal (BFS)

Both of these methods use the 'visited' flag.

difference between dfs and bfs

DFS (Depth-First Traversal)

In DFS, we process all of a vertex's descendants before we move to an adjacent vertex.

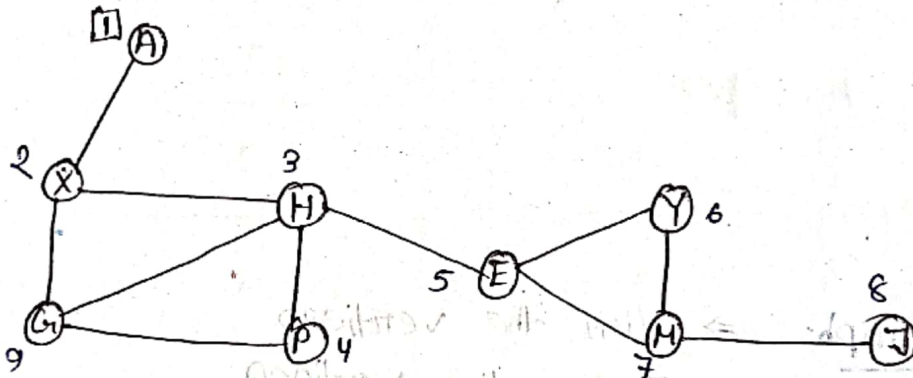


DFS: A B E F C D G H I

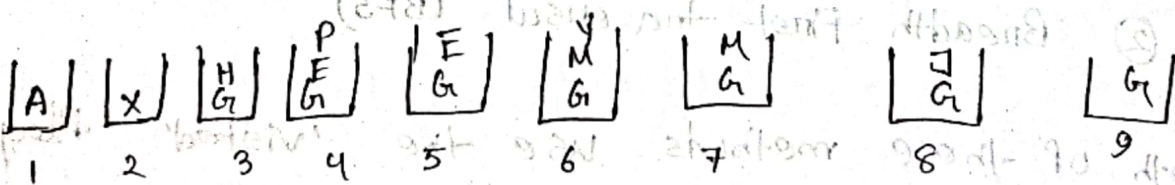
→ stack

→ Backtracking will be executed

Ex:



DFS: A X H P E Y M J G



stack contents

1. we begin by pushing the 1st vertex, A onto the stack

2. We then loop, POP the stack and after processing the vertex, push all of its adjacent vertices onto the stack.

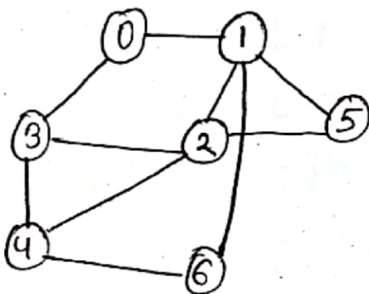
To process x at step 2 therefore, we pop x from the stack process it and then push u and v onto the stack.

3. when the stack is empty, the traversal is complete.

Example

Continue until all vertices reachable from the original source are discovered.

→ It's backtrack the stack to traverse all vertices

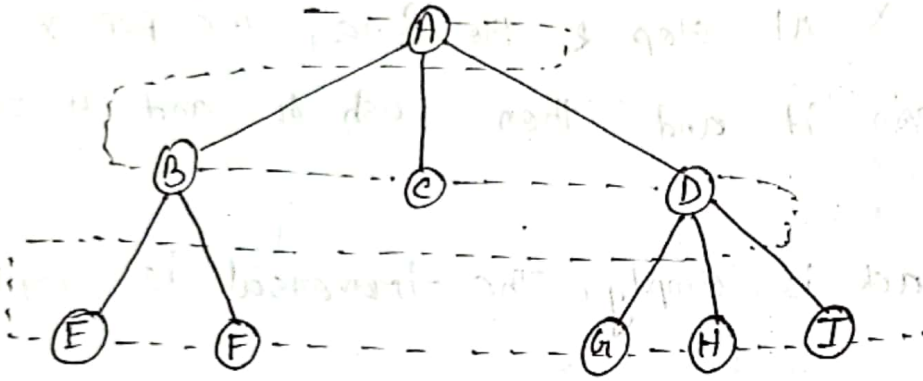


- Ⓐ stack: 0
- Ⓑ stack: 1 3 P: 0
- Ⓒ stack: 1 4 2 P: 0 3
- Ⓓ stack: 1 4 5 P: 0 3 2
- Ⓔ stack: 1 4 P: 0 3 2 5
- Ⓕ stack: 1 6 P: 0 3 2 5 4
- Ⓖ stack: 1 P: 0 3 2 5 4
- Ⓗ stack: $\emptyset$ P: 0 3 2 5 4 1



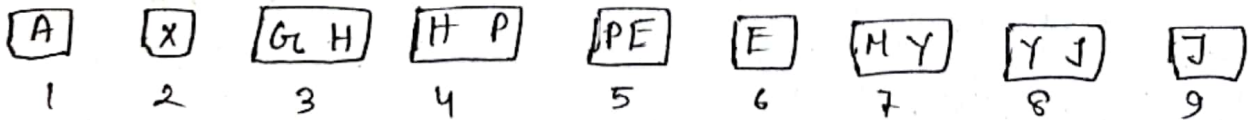
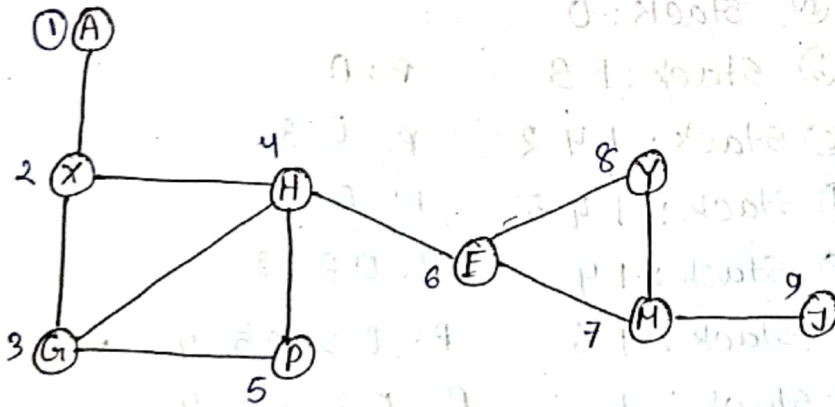
BFS (Breadth First Traversal)

In BFS, we process all adjacent vertices of a vertex before going to the next level. BFS uses queue.

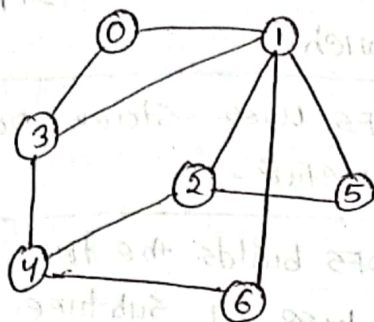


BFS: A B C D E F G H I

Example: 1



Example-2



Adjacency list

0: 1, 3

1: 3, 5, 2, 0

2: 1, 5, 4

3: 1, 0, 4

4: 3, 6, 2

5: 2, 1

6: 1, 4

Ⓐ QUEUE: 0

Ⓑ QUEUE: 0, 1, 3

Ⓒ QUEUE: 0, 1, 3, 5, 2

Ⓓ QUEUE: 0, 1, 3, 5, 2, 4

Ⓔ QUEUE: 0, 1, 3, 5, 2, 4

Ⓕ QUEUE: 0, 1, 3, 5, 2, 4

Ⓖ QUEUE: 0, 1, 3, 5, 2, 4, 6

Ⓗ QUEUE: 0, 1, 3, 5, 2, 4, 6

Print:

P: 0

P: 0, 1

P: 0, 1, 3

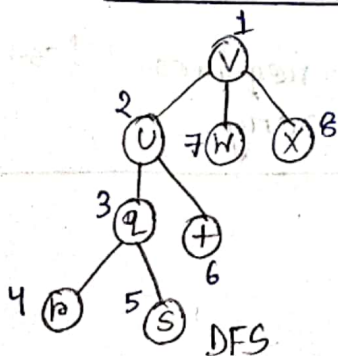
P: 0, 1, 3, 5

P: 0, 1, 3, 5, 2

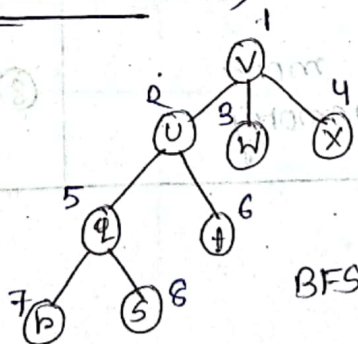
P: 0, 1, 3, 5, 2, 4

P: 0, 1, 3, 5, 2, 4, 6

Traversals Example (BFS and DFS)



DFS: V U q p s t W X



BFS: V U W X q t p s

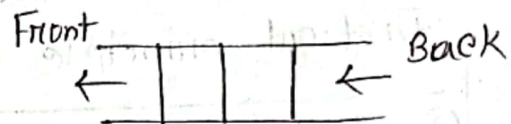
Difference between BFS and DFS:

BFS	DFS
① BFS stands for Breadth First Search	① DFS stands for Depth-First Search.
② BFS uses Queue Data Structure	② DFS uses stack Data Structure.
③ BFS builds the tree level by level	③ DFS builds the tree subtree by subtree
④ It works on the concept of FIFO	④ It works on the concept of LIFO
⑤ In BFS, there is no concept of backtracking	⑤ DFS algorithm uses the idea of backtracking
⑥ Application: Bipartite graphs, Shortest path etc	⑥ Application: Acyclic graphs, topological order etc.
⑦ BFS is slow compared to DFS	⑦ DFS is fast as compared to BFS.
⑧ BFS requires more memory	⑧ DFS requires less memory.

Difference between Stack and Queue;

Stack	Queue
① The objects are removed or inserted at the same end.	① The objects are removed and inserted from two different ends.
② It follows the Last In, First out principle (LIFO)	② It follows the First In First out (FIFO) principle.
③ Stack uses 'push' and 'pop' as two of its operation.	③ QUEUE uses enqueue and dequeue as two of its operation.
④ Insertion and deletion of elements take place from the TOP.	④ Insertion uses the rear end, and deletion uses the front end.
⑤ When $Top == max-1$, it means that the stack is full.	⑤ when $rear == max-1$, it means that queue is full.
⑥ stack does not have any types.	⑥ Queue has three types Circular Queue Priority Queue double ended Queue
⑦ The implementation is simpler in stack	⑦ The implementation is comparatively more complex in Queue.

Queue: A queue system is a linear list in which deletions can take place only at one end - the front of the list, and the insertions can take place only at the other end of the list - the 'back' is called a First in First out (FIFO)



⇒ One end is called front

⇒ One end is called rear

⇒ Additions (insertions or enqueue) are done at the rear only

⇒ Removals (deletions or dequeue) are made from the front only

Tree: A tree is an undirected graph which is connected and acyclic.

⇒ $G(V, E)$ is connected

⇒ $G(V, E)$ is acyclic

⇒ $|E| = |V| - 1$

Minimum spanning tree: A spanning tree is an undirected graph $G(V, E)$ is a subset of edges $T \subseteq E$ that are acyclic and connect all the vertices in V .

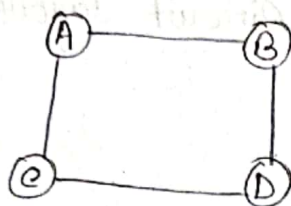
A connected subgraph S of graph $G(V, E)$ is said to be spanning if

⇒ S should contain all vertices of G

⇒ S should contain $(|V| - 1)$ edges.

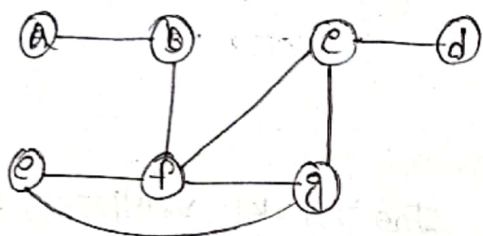
⇒ spanning tree must consists of exactly $n-1$ edges where vertex number is n

Example:

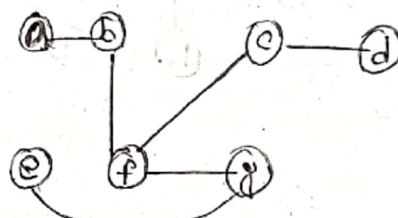


There will be no cycle in spanning tree.

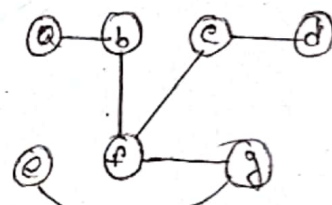
[A থেকে D / B → C connect হবে না, অগত্যা cycle তৈরি হবে না]



edge removed



e, f



c, g

node connected

□ How many ways to take a spanning tree from a complete graph?

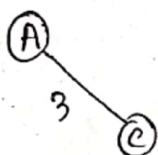
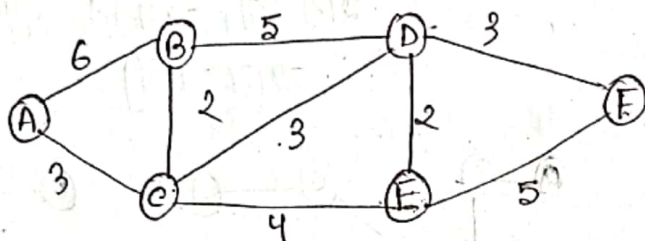
⇒ $n(n-2)$ where n = number of vertices

Minimum Spanning tree: A minimum spanning tree is a tree that contains all of the vertices in the graph and enough of its edges to form a tree.

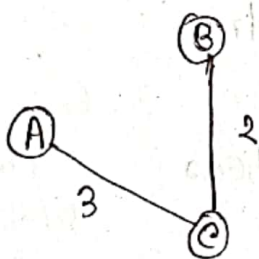
The minimum spanning tree of a network is a spanning tree in which the sum of its edge weights is guaranteed to be minimal.

Application of MST: $\Rightarrow$ Network designing
 $\Rightarrow$ Transportation planning
 $\Rightarrow$ Circuit layout

Example:



① Insert all vertices. $\textcircled{A}$
 we have vertices A and C in MST. $A \xrightarrow{6} B$ and $A \xrightarrow{3} C$.
 we choose the edge with the minimum weight that connects a vertex in MST.



② Now, our options are

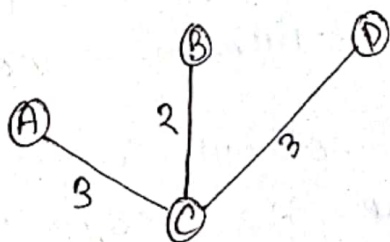
AB , weight = 6

$C \rightarrow E$, weight = 4

$C \rightarrow D$, weight = 3

$C \rightarrow B$, weight = 2 ✓

And we choose CB because it has the minimum weight.

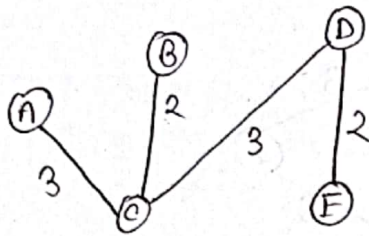


③ Now, $CD = 3$ ✓ weight

$CE = 4$

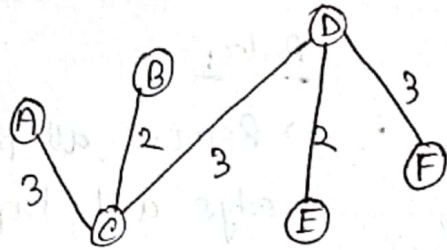
$B \rightarrow D$, weight = 5

And we choose ~~CB~~ CD



④ $CE = 4$
 $DE = 2$
 $DF = 3$

And we choose DE

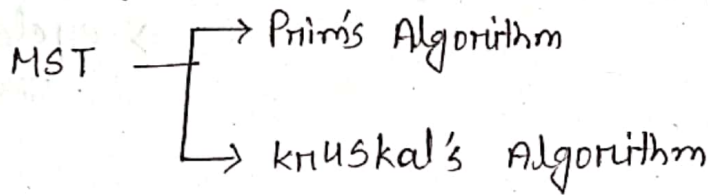


⑤ $EF = 5$
 $DF = 3$

And we choose DF

(Final spanning Tree)

Two ways to solve MST:



Prim's Algorithm:

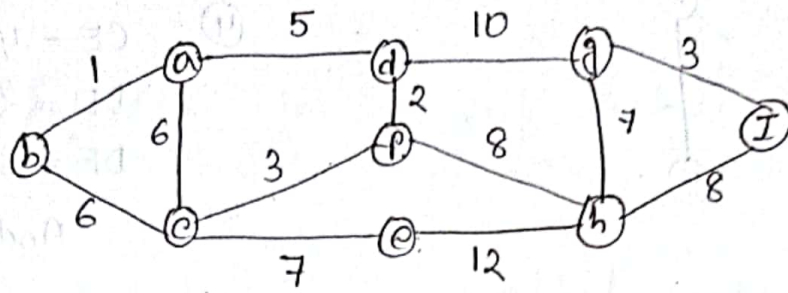
- ⇒ A greedy method to obtain a minimum cost spanning tree builds this tree edge by edge
- ⇒ The algorithm will start with a tree that includes only a minimum-cost edge of graph G.
- ⇒ The edges are added to this tree one by one.

* $V = (E+1)$
 ↗ Edge
 ↘ Vertices

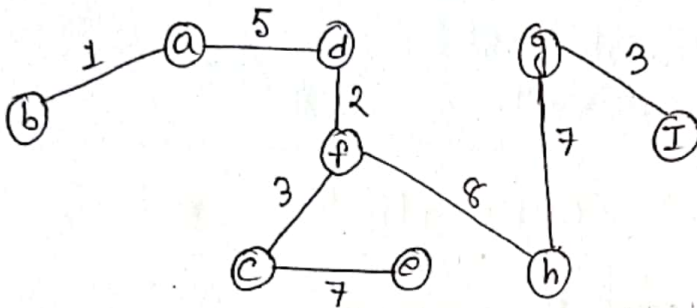
If ~~Edge~~ Edge is 5
 then vertices is $5+1=6$.

Prim's algorithm কে তথ্য always ওঠে
 check করতে হয়

Example:



Here, let b is a source node.



Rules:

⇒ Remove all parallel edge and loop edge

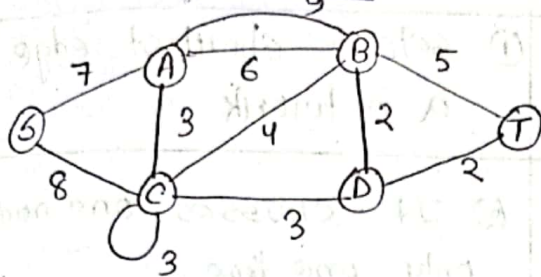
⇒ If source node দিয়া মাফত দাখে, তা মাফত মোফনো node টা source node সীমিত
⇒ cycle তরী বন্ধা মাফে না।

Prim's Algorithm

Explanation: b টা source node সীমিত নিশ্চি, b এর edge এর সীমিত minimum হাল 1 তাই $b \rightarrow a$ edge নিশ্চি. আর হাতে আছে 6. And a মার minimum edge হাল 5. আর হাতে আছে 6 তাই $a \rightarrow d$. বাকি থাকে 6, 6, d এর related edge হাল 10, 2, ওজন 10, 2, 6, 6 এর minimum 2. তাই $d \rightarrow f$ হাতে 10, 6, 6. f এর adjacency edge 3, 8, 3, 8, 10, 6, 6 এর সীমিত 3 minimum. তাই $f \rightarrow c$ হাতে 8, 10, 6, 6. c এর adjacency 6, 6, 7, 10, 6, 8, 6, 7 এর সীমিত 6 minimum কিন্তু 6, 6 এর ত্যাগ cycle হাল মাফ তাই 7 নিত হাল, তাই $c \rightarrow e$, হাতে 10, 8, e এর adjacency 12, 10, 6, 12 এর minimum 8, তাই $e \rightarrow h$ হাতে 12, 10, 8, h এর adjacency 7, 8, 7, 8, 8 এর min 7.

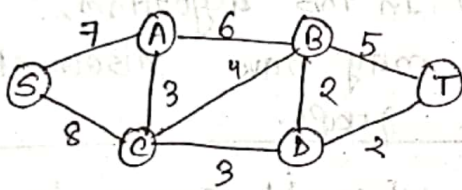
ଆଉ (H) ଓ (I) : (I) ବା adjacency 3, 8, 8 (I) ବା
 min 3, ଆଉ (I) ଓ (I)

Kruskal's Algorithm:



[Cycle ଚେକି ନା]

Step-1 Remove all parallel edge and loop edge

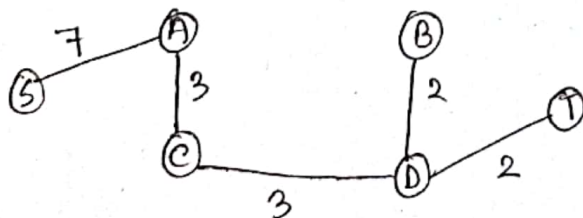


Step-2 Sort the edges into ascending order by weight

B, D	D, T	A, C	C, D	B, C	B, T	A, B	S, A	S, C
2	2	3	3	4	5	6	7	8
				X	X	X		X

Step-3

ଏକ vertex ଓ ଆକାଞ୍ଚ ସେ
 ନିଅ ନିଅ,



It is minimal spanning tree

Difference between Prim's and Kruskal's Algorithm:

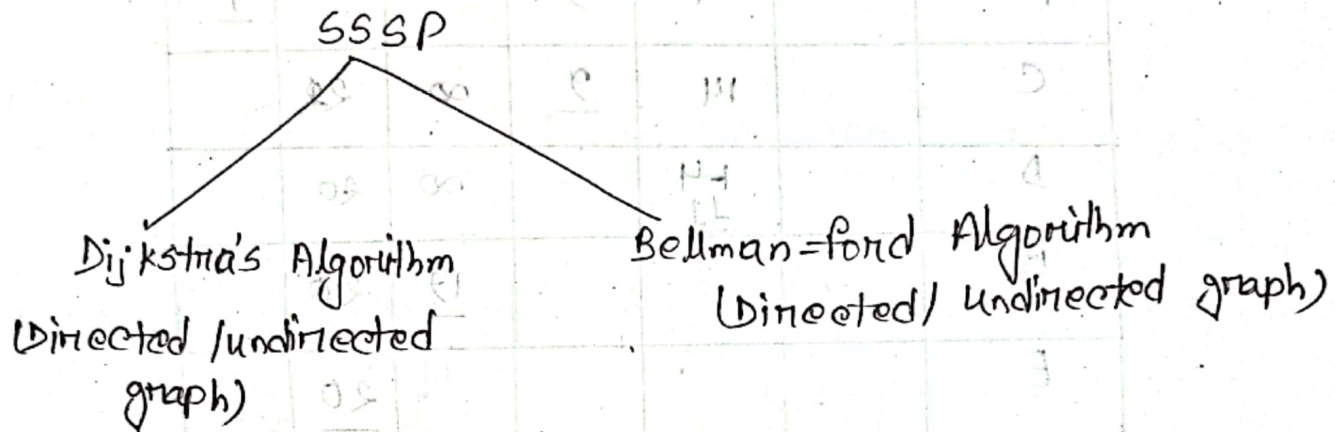
Prim's Algorithm	Kruskal's Algorithm
① select any vertex	① select shortest edge in a network
② It traverses one node more than one time.	② It crosses one node only one time
③ Time complexity: $O(V^2)$	③ Time complexity: $O(E \log V)$
④ In this algorithm, all the elements must be connected.	④ In this algorithm, may have disconnected graph.
⑤ When it comes to dense graphs, the Prim's algorithm runs faster.	⑤ When it comes to sparse graphs, Kruskal's algorithm runs faster.
⑥ It prefers list data structure	⑥ It prefers the heap data structure.



Segment - 6 (Shortest path)

Single source shortest path:

It finds the shortest path from a single source vertex to all other vertices in a weighted graph.

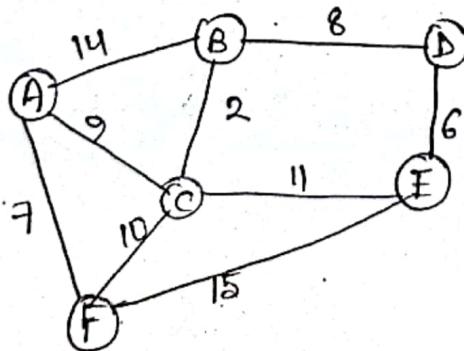


Dijkstra's Algorithm:

Relaxation: If $(d(u) + e(u, v)) < d(v)$
 $d(v) = d(u) + e(u, v)$

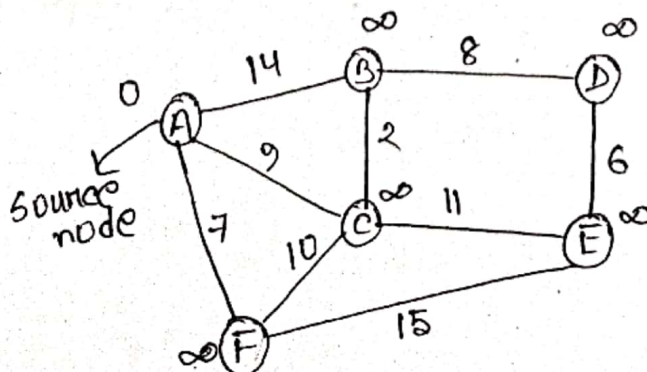
$d(u)$ - vertex
 $e(u, v)$ - edge weight

Example:



Soln:

Step-1



Source node 0 चिह्नित
बाकि node ∞ चिह्नित

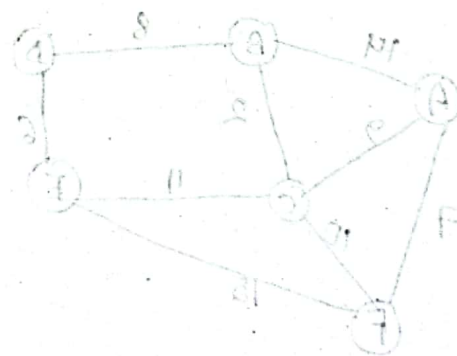
(dibag khatmal) 3- tumpak

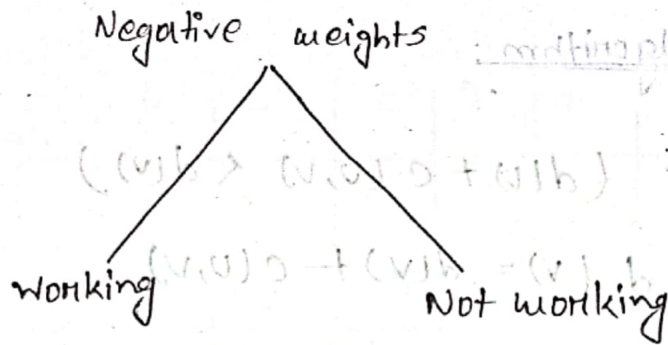
Step-2

Visited vertex	A	B	C	D	E	F
A	<u>0</u>	∞	∞	∞	∞	∞
F		14	9	∞	∞	<u>7</u>
C		14	<u>9</u>	∞	22	
D		<u>14</u>		∞	20	
B				<u>19</u>	20	
E					<u>20</u>	

$A \rightarrow C \rightarrow E$

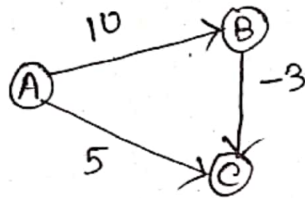
[Backtracking ব্যবস্থা]
মেথালো vale change হয়
ব vertex বিত হয়]





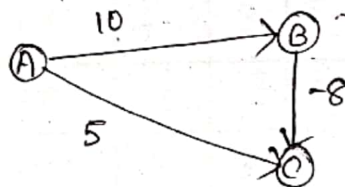
Q Dijkstra's Algorithm doesn't always work on negative weights.

Working:



Visted vertex	A	B	C
A	0	∞	∞
C		10	5
B		10	

Not working:



Visted vertices	A	B	C
A	0	∞	∞
B		∞	5
C			

Time complexity: min heap - heap sort $O(n \log n)$
 $O(E \log V)$

→ Dijkstra's Algorithm is a greedy Algorithm

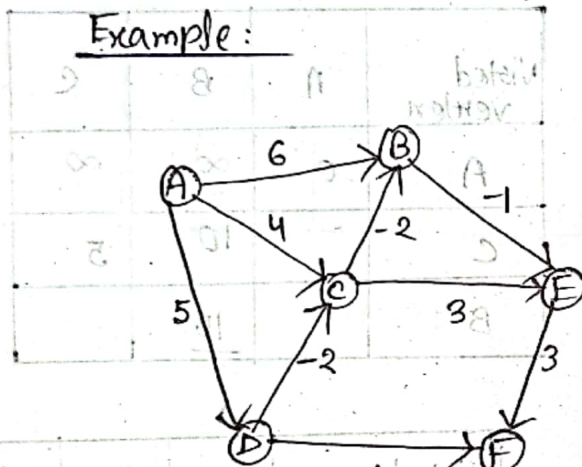
Bell-man -ford Algorithm:

Relaxation: if $(d(u) + c(u,v) < d(v))$

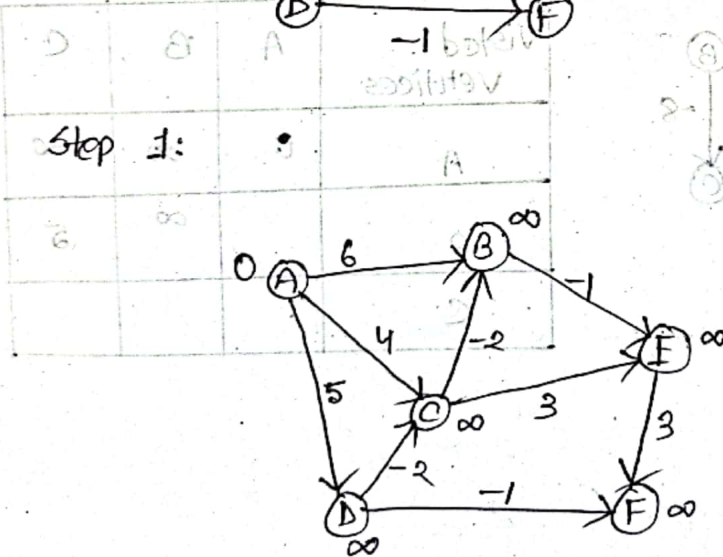
$$d(v) = d(u) + c(u,v)$$

Iteration: $(n-1)$ [n = no of vertices]

Example:



Step 1:



Source node = 0

যাকিগুলো ∞ বৈতে হয়,

Step-2

1st Iteration

Visited vertices	A	B	C	D	E	F
A	<u>0</u>	∞	∞	∞	∞	∞
B	0	<u>6</u>	4	5	∞	∞
C	0	6	<u>4</u>	5	5	∞
D	0	2	4	<u>5</u>	5	∞
E	0	2	3	5	<u>5</u>	4
F	0	2	3	5	5	<u>4</u>

2nd iteration:

Visited vertices	A	B	C	D	E	F
A	<u>0</u>	2	3	5	5	4
B	0	<u>2</u>	3	5	5	4
C	0	2	3	5	<u>1</u>	4
D	0	1	3	5	1	4
E	0	1	3	5	1	4
F	0	1	3	5	1	4

3rd Iteration:

Visited vertices	A	B	C	D	E	F
A	0	1	3	5	1	4
B	0	<u>1</u>	3	5	1	4
C	0	1	3	5	0	4
D	0	1	3	5	0	4
E	0	1	3	5	0	4
F	0	1	3	5	0	3

4th iteration:

Visited Vertices	A	B	C	D	E	F
A	0	1	3	5	0	3
B	0	1	3	5	0	3
C	0	1	3	5	0	3
D	0	1	3	5	0	3
E	0	1	3	5	0	3
F	0	1	3	5	0	3

দুইটা iteration
এর value same
হলে থামতে
হয়।

we will stop the iteration.

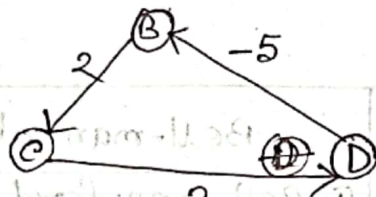
Answer:

	A	B	C	D	E	F
A	0	1	3	5	0	3
B	0	1	3	5	0	3
C	0	1	3	5	0	3
D	0	1	3	5	0	3
E	0	1	3	5	0	3
F	0	1	3	5	0	3

	A	B	C	D	E	F
A	0	1	3	5	0	3
B	0	1	3	5	0	3
C	0	1	3	5	0	3
D	0	1	3	5	0	3
E	0	1	3	5	0	3
F	0	1	3	5	0	3

Drawbacks of Bellman-ford Algorithm:

Can't work on negative cycle.



1st Iteration:

	B	C	D
A	∞	0	∞
B	∞	0	2
C	-3	0	2
D	-3	-1	2

2nd Iteration:

	B	C	D
B	-3	-1	2
C	-3	-1	2
D	-3	-1	1
	-4	-1	1

For 3 vertex iteration will be 2. Here after 2 iteration the value was still changing. So, we can see that Bell-ford man algorithm is not applicable is negative cycle.

Time complexity: $O(V^2)$

Difference between Dijkstra's Algorithm and Bellman-Ford algorithm:

Dijkstra's Algorithm	Bellman-Ford algorithm
① Dijkstra's algorithm may or may not work when there is negative weight edge.	① Bellman-Ford algorithm works when there is negative weight edge.
② Time complexity $O(E \log V)$	② Time complexity $O(VE)$
③ It is less time consuming.	③ It is more time consuming than Dijkstra's algorithm.
④ It cannot be implemented easily in a distributed way.	④ It can easily be implemented in a distributed way.
⑤ Greedy approach is taken to implement the algorithm.	⑤ Dynamic programming approach is taken to implement the algorithm.
⑥ It has less overhead than Bellman algorithm.	⑥ It has more overheads than Dijkstra's algorithm.

Floyd warshalls Algorithm

The resulting algorithm known as the Floyd warshall Algorithm, runs in $O(V^3)$ time.

use a dynamic programming formulation to solve the all-pairs shortest path problem on a directed graph $G(V, E)$

⇒ There are no negative weight cycle.

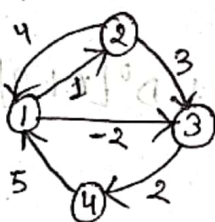
⇒ It doesn't works on negative cycle.

Formula:
$$D_{ij}^k = \begin{cases} w_{ij} & \text{if } k=0 \\ \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) & \text{if } k \geq 1 \end{cases}$$

$$\infty_{ij}^k = \begin{cases} \infty & \text{if } i=j \text{ or } w_{ij} = \infty \\ i & \text{if } i \neq j \text{ and } w_{ij} < \infty \end{cases}$$

$$\rightarrow D^k[i, j] = \min \{ D^{k-1}[i, j], D^{k-1}[i, k] + D^{k-1}[k, j] \}$$

Example:



$$D^0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 1 & -2 & \infty \\ 4 & 0 & 3 & \infty \\ \infty & \infty & 0 & 2 \\ 5 & \infty & \infty & 0 \end{bmatrix} \end{matrix}$$

$$D^1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 1 & -2 & \infty \\ 4 & 0 & 2 & \infty \\ \infty & \infty & 0 & 2 \\ 5 & 6 & 3 & 0 \end{bmatrix} \end{matrix}$$

For, $D^1, k=1$

$$\begin{aligned} D^1[2, 3] &= \min \{ D^{0-1}[2, 3], D^{1-1}[2, 1] + D^{1-1}[1, 3] \} \\ &= \min \{ D^0[2, 3], D^0[2, 1] + D^0[1, 3] \} \\ &= \min \{ 3, 4 + (-2) \} \\ &= \min \{ 3, 2 \} \\ &= 2 \end{aligned}$$

Minimum Value 2

$$\begin{aligned}
 D^1[2,4] &= \min \{ D^0[2,4], D^0[2,1] + D^0[1,4] \} \\
 &= \min \{ \infty, 4 + \infty \} \\
 &= \min \{ \infty, \infty \} \\
 &= \infty
 \end{aligned}$$

$$\begin{aligned}
 D^1[3,2] &= \min \{ D^0[3,2], D^0[3,1] + D^0[1,2] \} \\
 &= \min \{ \infty, \infty + 1 \} \\
 &= \min \{ \infty, \infty \} \\
 &= \infty
 \end{aligned}$$

$$\begin{aligned}
 D^1[3,4] &= \min \{ D^0[3,4], D^0[3,1] + D^0[1,4] \} \\
 &= \min \{ 2, \infty + \infty \} \\
 &= \min \{ 2, \infty \} \\
 &= 2
 \end{aligned}$$

$$\begin{aligned}
 D^1[4,2] &= \min \{ D^0[4,2], D^0[4,1] + D^0[1,2] \} \\
 &= \min \{ \infty, 5 + 1 \} \\
 &= \min \{ \infty, 6 \} \\
 &= 6
 \end{aligned}$$

$$\begin{aligned}
 D^1[4,3] &= \min \{ D^0[4,3], D^0[4,1] + D^0[1,3] \} \\
 &= \min \{ \infty, 5 + (-2) \} = \min \{ \infty, 3 \} = 3
 \end{aligned}$$

Now,

$$D^2 = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 0 & 1 & -2 & \infty \\ 4 & 0 & 2 & \infty \\ \infty & \infty & 0 & 2 \\ 5 & 6 & 3 & 0 \end{bmatrix}$$

Here, $k=2$

$$\begin{aligned}
 D^2[1,3] &= \min \{ D^{2-1}[1,3], D^{2-1}[1,2] + D^{2-1}[2,3] \} \\
 &= \min \{ D^1[1,3], D^1[1,2] + D^1[2,3] \} \\
 &= \min \{ -2, 1 + 2 \} = \min \{ -2, 3 \} = -2
 \end{aligned}$$

$$\begin{aligned}
 D^2[1,4] &= \min \{ D^1[1,4], D^1[1,2] + D^1[2,4] \} \\
 &= \min \{ \infty, 1 + \infty \} = \min \{ \infty, \infty \} = \infty
 \end{aligned}$$

$$D^2[3,1] = \min \{ D^1[3,1], D^1[3,2] + D^1[2,1] \}$$

$$= \min \{ \infty, \infty + 4 \} = \{ \infty, \infty \} = \infty$$

$$D^2[3,4] = \min \{ D^1[3,4], D^1[3,2] + D^1[2,4] \}$$

$$= \min \{ 2, \infty + \infty \} = \{ 2, \infty \} = 2$$

$$D^2[4,1] = \min \{ D^1[4,1], D^1[4,2] + D^1[2,1] \}$$

$$= \min \{ 5, 6 + 4 \} = \min \{ 5, 10 \} = 5$$

$$D^2[4,3] = \min \{ D^1[4,3], D^1[4,2] + D^1[2,3] \}$$

$$= \min \{ 3, 6 + 2 \} = \min \{ 3, 8 \} = 3$$

Now, $D^3 =$

	1	2	3	4
1	0	1	-2	0
2	4	0	2	4
3	∞	∞	0	2
4	5	6	3	0

Hence, $k=3$,

$$D^3[1,2] = \min \{ D^{3-1}[1,2], D^{3-1}[1,3] + D^{3-1}[3,2] \}$$

$$= \min \{ D^2[1,2], D^2[1,3] + D^2[3,2] \}$$

$$= \min \{ 1, -2 + \infty \} = \{ -1, \infty \} = -1$$

$$D^3[1,4] = \min \{ D^2[1,4], D^2[1,3] + D^2[3,4] \}$$

$$= \min \{ \infty, -2 + 2 \} = \{ \infty, 0 \} = 0$$

$$D^3[2,1] = \min \{ D^2[2,1], D^2[2,3] + D^2[3,1] \}$$

$$= \min \{ 4, 2 + \infty \} = \{ 4, \infty \} = 4$$

$$D^3[2,4] = \min \{ D^2[2,4], D^2[2,3] + D^2[3,4] \}$$

$$= \min \{ \infty, 2 + 2 \} = \{ \infty, 4 \} = 4$$

$$D^3[4,1] = \min \{ D^2[4,1], D^2[4,3] + D^2[3,1] \}$$

$$= \min \{ 5, 3 + \infty \} = \{ 5, \infty \} = 5$$

$$D^3[4,2] = \min \{ D^2[4,2], D^2[4,3] + D^2[3,2] \}$$

$$= \min \{ 6, 3 + \infty \} = \{ 6, \infty \} = 6$$

$D^4 = \begin{matrix} & 1 & 2 & 3 & 4 \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 1 & -2 & \infty \\ 4 & 0 & 2 & \infty \\ 7 & 8 & 0 & \infty \\ 5 & 6 & 3 & 0 \end{bmatrix} \end{matrix}$

$$D^4[1,3] = \min \{ D^3[1,3], D^3[1,4] + D^3[4,3] \}$$

$$= \min \{ -2, 0 + 3 \} = \min \{ -2, 3 \} = -2$$

$$D^4[2,1] = \min \{ D^3[2,1], D^3[2,4] + D^3[4,1] \}$$

$$= \min \{ 4, 4 + 5 \} = \min \{ 4, 9 \} = 4$$

$$D^4[2,3] = \min \{ D^3[2,3], D^3[2,4] + D^3[4,3] \}$$

$$= \min \{ 2, 4 + 3 \} = \min \{ 2, 7 \} = 2$$

$$D^4[3,1] = \min \{ D^3[3,1], D^3[3,4] + D^3[4,1] \}$$

$$= \min \{ \infty, 2 + 5 \} = \min \{ \infty, 7 \} = 7$$

$$D^4[3,2] = \min \{ D^3[3,2], D^3[3,4] + D^3[4,2] \}$$

$$= \min \{ \infty, 2 + 6 \} = \min \{ \infty, 8 \} = 8$$